

Route Optimization Model - Implementation Report

Project Objective

- Build an end-to-end route optimizer using graph algorithms (Dijkstra) on Indian road/rail network.
- Add AI-generated natural language route explanations.
- Provide interactive map visualization and route overlays.

Backend Delivered (FastAPI)

- API endpoints: GET /api/graph and POST /api/route.
- Graph engine supports optimization by time, cost, and distance.
- Route modes: road only, rail only, and road+rail.
- Segment-wise output with totals (time, INR cost, distance).
- Rule-based alerts for major NH advisory patterns.
- Cargo intelligence advisory module integrated.

Graph & Routing Core

- Implemented weighted graph traversal with Dijkstra.
- Unified adjacency representation for road and rail edges.
- Dynamic objective switching at runtime:
 - * time -> fastest route
 - * cost -> cheapest route (toll/fare)
 - * km -> shortest route
- Supports mixed-mode pathfinding in road_rail mode.

Data Layer

- Added demo 21-city India network model with road + rail edges.
- Edge schema includes: id, from_city, to_city, type, label, distance_km, time_hours, cost_inr.
- City coordinate dictionaries prepared for map rendering.

Cargo System Upgrade (as requested)

- Replaced simple cargo list with 10 categorized cargo groups and sub-types:
 - 1) Perishable Goods
 - 2) Non-Perishable Goods
 - 3) Fragile Goods
 - 4) Hazardous Materials (Hazmat)
 - 5) Livestock
 - 6) Heavy / Bulk Cargo
 - 7) High-Value Goods
 - 8) General Cargo
 - 9) Oversized Cargo
 - 10) Liquid Cargo
- Frontend sends cargo as "Category|Item"; backend parses and tailors advisory notes.

AI Route Briefing

- Added Claude integration via Anthropic Messages API.
- Prompt grounded on computed route facts only (anti-hallucination instruction).
- Added deterministic fallback explanation when API key is absent/unavailable.

ML Component

- Added optional ML weight adjuster module for time/cost multipliers.
- Synthetic training pipeline included so feature works out-of-the-box.
- Artifact schema checks and retraining guard implemented for feature mismatch recovery.
- Updated ML cargo handling to align with new category format.

Map Integration Evolution

- Replaced custom SVG network panel with real map container.
- Google Maps route rendering integrated (when key is available).
- Added no-key fallback path:
 - * OpenStreetMap tiles via Leaflet
 - * OSRM routing for drivable paths
 - * polyline fallback when OSRM is unavailable or for rail-only visualization

Frontend Delivered

- Route builder panel with origin, destination, mode, optimize-by, cargo type, ML toggle, month.
- API response rendering: explanation, totals, alerts, cargo notes, and segment table.
- Automatic map provider behavior:
 - * If GMAPS_API_KEY exists -> Google Maps
 - * Else -> OpenStreetMap fallback (no API key)

Reliability & Debugging Work Completed

- Fixed backend reload crash caused by stale import after cargo module refactor.
- Fixed ML artifact feature-length mismatch handling.
- Corrected route behavior for Nashik->Pune timing example in demo data.
- Preserved API compatibility while extending cargo and map functionality.

Key Files Created/Updated

- backend/main.py, backend/router.py, backend/graph_data.py
- backend/alerts.py, backend/cargo.py, backend/lm.py, backend/ml_weight_adjuster.py
- frontend/index.html, frontend/app.js, frontend/styles.css, frontend/config.js
- README.md, .env.example, requirements.txt

How to Run

1. Start server: `python -m uvicorn backend.main:app --reload --port 8000`
2. Open: `http://127.0.0.1:8000`
3. Optional Google mode: set `frontend/config.js` -> `window.GMAPS_API_KEY="YOUR_KEY"`
4. Without key, OpenStreetMap fallback is used automatically.

Current Status

- End-to-end MVP is implemented and operational.
- Supports routing, cargo advisories, alerts, AI explanation fallback, and map visualization.